

What Actually Generalizes: Data Coverage for Multi-Platform Inertial Velocity Estimation

Kaggle team name: CoCEL @POSTECH

Sanghyun Park^{1*}, Seongjun Kim^{2*}, and Soohye Han^{1*}, *Senior Member, IEEE*

Abstract—This report describes our entry to the IMU Odometry Challenge: from a 1-s window of a 200 Hz 6-axis IMU, predict the window’s mean body-frame velocity across four platforms (car, legged, drone and human) with one shared set of weights. Two findings shaped the work. Differentiating position into velocity amplifies its error, and relocalisation events in the ground-truth pipeline leave discrete jumps that the 1-s average does not fully remove. More importantly, the data’s coverage is uneven: many drone sequences exceed 10 m/s while the other platforms stay below 5 m/s once averaged, so the drone dominates the error. We therefore read the data, not the architecture, as the main bottleneck. Our model is a two-branch ConvNeXt-V2 encoder whose raw-input branch is modulated by FiLM from a second branch that reads a moving average of the accelerometer, a proxy for body-frame gravity, followed by a bi-GRU and three heads combined by a mixture of experts. It has 8.03 M parameters, is trained in a single run from random initialisation on the official train and val splits and their augmented copies with no external data, and infers all 89 test sequences in 15.18 s. On the official scoring service it reaches a macro score of 0.2087 (AVE 0.1688 m/s, ATE₂₀ 0.5533 m), and 0.25392 on the public leaderboard, ranked 5th.

I. INTRODUCTION

An IMU needs neither external infrastructure nor a line of sight, which makes it the only proprioceptive sensor left where vision and LiDAR cannot sense enough of the environment for odometry to succeed. Strapdown integration, however, integrates bias twice, and the position error grows with the cube of time. Learned inertial odometry bypasses that integration and *regresses velocity or displacement directly*: the polar displacement of IONet [1], the velocity regression of RoNIN [2], TLIO [3], which turns uncertainty into a filter measurement, and IDOL [4], which solves orientation first, all belong to that family. They rest on one assumption: that *only one kind of motion* is ever seen.

That assumption is what an inertial foundation model has to break, and TartanIMU [5] proposed handling four embodiments with a single light model. This challenge pins that down: from a 1-s window of a 200 Hz 6-axis IMU, predict the window’s mean body-frame velocity with **one shared set of**

weights, on an evaluation split that is anonymised so that no label exists at inference time. Because the score weights the four platforms equally, differences in window count are not compensated, and in our error distribution the aerial platform alone accounts for 61.7 % of the score. The difficulty lies not in the average but in **improving the hardest platform without hurting the rest**.

Through this challenge we made two findings. First, differentiating position into velocity amplifies its error. Averaging over one second mitigates most of that noise, but the ground truth still has some inaccurate regions, which we traced to relocalisation events in the ground-truth generation pipeline that leave discrete jumps in the label. Second, the data’s coverage is insufficient. As noted above, the drone accounts for most of the error: many of its sequences exceed 10 m/s, while the other three platforms never exceed 5 m/s once averaged over one second. The drone therefore has to fit a wider range, and because that range is larger in scale, its error is inevitably larger too. We therefore believe the main bottleneck of inertial odometry is the data.

Even so, we still had to raise generalisation performance with the data we were given, and made the following attempts. Our main contributions are therefore two-fold. (1) We noticed that body-frame gravity improves generalisation, so we use a moving average of the acceleration as a short-window estimate of gravity and feed it as an input. (2) That moving average lies in the low-frequency band, where most of an IMU’s motion is also concentrated; through FiLM, we let this low-frequency branch modulate the raw-input branch.

II. METHOD

A. Objective

The task is to use *only* the raw 6-axis IMU stream (T seconds at 200 Hz, $\{\mathbf{a}, \mathbf{w}\} \in \mathbb{R}^{B \times 200T \times 6}$, where B is the batch size and the left superscript B denotes the body frame), to predict the mean body-frame velocity of every 1-s window, $\mathbf{v} \in \mathbb{R}^{B \times T \times 3}$. Predicting only a 1-s mean, however, generally leaves the estimator short of supervision: the label arrives once per second, so the motion *between* labels is never constrained and is easily mispredicted. We therefore do not emit one output per second but a 12 Hz sequence, so that supervision also reaches the intervals between the 1-s labels. Passing through our network, the signal is downsampled four times in total to a fine-resolution 12 Hz output $\mathbf{v}_{\text{fine}} \in \mathbb{R}^{B \times 12T \times 3}$. Since 12 Hz has no

*All authors contributed equally to this submission.

¹The authors are with the Computational Control Engineering Laboratory (CoCEL), Department of Convergence IT Engineering, Pohang University of Science and Technology (POSTECH), Pohang 37673, South Korea. {pash0302, soohye.han}@postech.ac.kr

²Seongjun Kim is with the Computational Control Engineering Laboratory (CoCEL), Department of Electrical Engineering, Pohang University of Science and Technology (POSTECH), Pohang 37673, South Korea. seongjun4527@postech.ac.kr

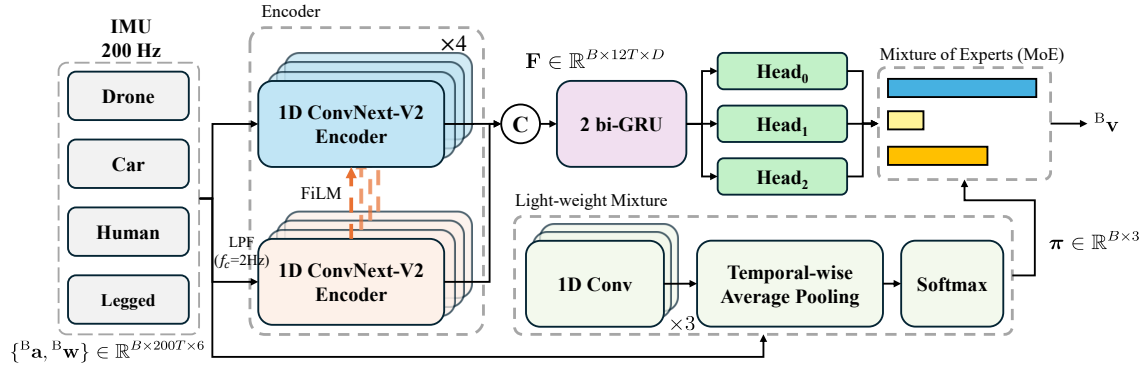


Fig. 1. Signal path of the network from input to output. The main branch (top) reads the raw IMU directly; the conditioning branch (bottom) reads only the accelerometer after a 0.5 s moving average and modulates the main branch by FiLM (dashed) at four depths. The two branches are concatenated and fed to a 2-layer bi-GRU, whose output is split across three heads. In parallel, a light-weight gate reads the raw input through a 1-D convolution, a temporal-wise average pool and a softmax, and emits a per-chunk confidence π over the three *anonymous* experts; the gate never sees a platform label. The output is the body-frame velocity, the soft mixture of the three heads.

exact representation on the discrete 200 Hz grid, the fine predictions are made at alternating strides of 16 and 17 samples, and the final 1-s mean is the stride-weighted average

$$\mathbf{B}_V = \frac{1}{200} \sum_{i=1}^{12} s_i \mathbf{B}_{V_{\text{fine}, i}}, \quad s_i \in \{16, 17\}, \quad (1)$$

which has no trainable parameter.

B. Network Design

Overview. Our architecture is depicted in Fig. 1. A batch is assembled at 200 Hz from the four platform datasets (drone, car, human and legged), and the network is built from three parts: a ConvNeXt-based encoder, a light-weight mixture, and multiple heads combined by a mixture of experts (MoE) [7].

Having seen AirIO [8] raise accuracy by encoding attitude, we ran further experiments on what other features could play that role, and found that feeding the body-frame gravity as an input raises accuracy as well. Since the body-frame gravity cannot be used as an input directly, we added a moving average of the body-frame accelerometer as a way of describing it; seen from another angle, this works because most of the motion energy is concentrated at low frequency. After the encoder the features pass through a bi-GRU, and, as in TartanIMU [5], prediction is made by multiple heads; to consolidate those heads into a single output we use an MoE structure.

Encoder and gravity conditioning. AirIO [8] uses plain convolutions alone, and TartanIMU [5] builds its encoder on ResNet. We instead note that in computer vision ConvNeXt [9] reaches distant points of the input with convolutions alone, much as a transformer does, and we therefore build our layers from its more advanced version, ConvNeXt-V2 [10]. Every pass through a ConvNeXt-V2 stage halves the temporal resolution, and in the downsampling layers we use group normalisation where the original design uses layer normalisation.

As stated above, we describe the body-frame gravity with a moving average, and noted that this component exposes most of the motion. We judge it to be a strong signal, and to

let it act on the encoder through which the raw data passes we adopt feature-wise linear modulation (FiLM) [6]: a pair (γ_c, β_c) is produced per channel and applied to every feature $F_{i,c}$ as

$$\text{FiLM}(F_{i,c} | \gamma_c, \beta_c) = (1 + \gamma_c) \cdot F_{i,c} + \beta_c. \quad (2)$$

We then concatenate the raw features with those of the moving-average branch and project the result down to a smaller scale. Finally, to model the temporal information we use a bi-GRU [11]. Models such as the transformer and Mamba have brought large advances to time-series modelling, but our sequence is not long ($T = 10$), so we use the bi-GRU alone. We write its output as $\mathbf{F}'$.

Multi-head mixture of experts. As in TartanIMU [5] we use multiple heads, and we bind them into a single model with an MoE structure. TartanIMU places one head per platform and therefore has four; we use three. The reason is that the legged platform and the car have very low degrees of freedom and travel mainly along x , so the two are meant to share one expert. A confidence $\pi \in \mathbb{R}^3$ over the experts then forms the final output

$$\mathbf{B}_V = \sum_i \pi_i \text{Head}_i(\mathbf{F}'). \quad (3)$$

We write it π rather than w so that it is never confused with the gyroscope $\mathbf{B}_W$. To learn π we use a convolution-based module, which here is not ConvNeXt-based but a light-weight convolution: a 1-D convolution over the raw input, a temporal-wise average pool, and a softmax.

Loss function design. The 12 Hz target labels are obtained by averaging the ground-truth body-frame velocity over the same 12 Hz blocks. Because the objective of this challenge is the 1-s mean, we additionally place a loss on the 1-s mean of our 12 Hz output; rather than averaging once per second, that is over 12 consecutive steps, we take a 1-s mean every three steps, which acts as a smoother supervision. The 1-s mean the challenge asks for is

$$\mathbf{B}_{V_{\text{avg}}} = \frac{1}{200} \sum_{i=1}^{200} \mathbf{B}_{V_i}. \quad (4)$$

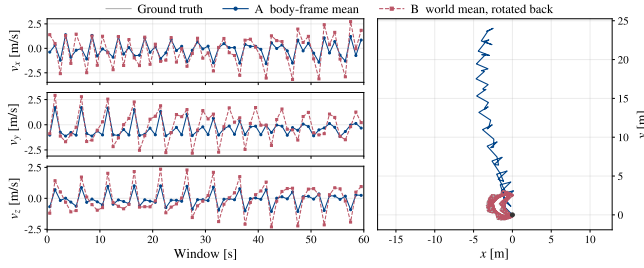


Fig. 2. Why the 1-s term is taken in the world frame. **A** is the window mean formed directly in the body frame, as in (4); **B** is the mean formed in the world frame and rotated back at the end. *Left*: the three velocity components of both against the ground truth. *Right*: the path obtained by integrating each. Where the rotation repeats within a window the two are no longer the same quantity, and the body-frame mean stops describing the pose.

In effect, however, this averages the rotation as well; rewriting (4),

$$\mathbf{B}\mathbf{v}_{\text{avg}} = \frac{1}{200} \sum_{i=1}^{200} \mathbf{R}_i^\top \mathbf{w}_i \mathbf{v}_i, \quad (5)$$

and, as Fig. 2 shows, this can fail to describe the pose when extreme rotations repeat. We therefore define the 1-s mean in the world frame when forming the loss. This choice is not free: under our L1 loss, the world-frame and the body-frame loss are not the same term, which we show formally in Appendix E.

One could make the grouping of the legged platform and the car explicit and train the gate with a cross-entropy loss against that partition, but a hand-made partition fixes the boundary in advance and feeds platform labels to the router. We use entropy terms instead (§III-A). They prevent *expert collapse*, in which an expert selected slightly more often early receives more gradient, is then selected still more, and finally saturates the softmax so that the routing freezes. A commitment term whose sign is ramped from negative to positive keeps the assignment soft early and lets the experts specialise later, and the experts stay anonymous: no platform label enters the gate.

The final loss is therefore

$$\mathcal{L} = \lambda_{\text{vel}} \mathcal{L}_{\text{vel}} + \lambda_{\text{window}} \mathcal{L}_{\text{window}} + \lambda_{\text{entropy}} \mathcal{L}_{\text{entropy}}, \quad (6)$$

where $\mathcal{L}_{\text{vel}}$ is the loss on the 12 Hz fine predicted velocity, $\mathcal{L}_{\text{window}}$ the loss on the 1-s mean, and $\mathcal{L}_{\text{entropy}}$ the entropy loss on the confidence of each expert.

C. Data Processing

We use the raw IMU data as it is. The 0.5-s moving average of §II-B is likewise a single pooling operation, `torch.nn.AvgPool1d(kernel_size=101, stride=1)` over the accelerometer channels, and therefore carries no learnable parameter.

Infeasible label processing. We analysed the ground-truth labels. In these datasets the ground truth is produced by differentiating the pose at 200 Hz and rotating the result into the body frame. As noted above, most of the noise this introduces is washed out once the prediction is averaged over 1 s. Even so, we found a number of infeasible labels, which we either masked out during training or, over short intervals,

replaced by a pseudo ground truth obtained from a zero-velocity update (ZUPT) or a motion model. Two cases are shown for the car platform in Fig. 4. In (a), the released speed of `car_val_0007` reaches values above 20 m/s that survive the 1-s mean, and we masked the windows concerned. In (b), the label of `car_val_0008` jumps by more than 100 m/s between consecutive samples; inspecting the position shows a long gap in the track, preceded by motion at constant velocity. We presume that no signal was received over that stretch, so localisation failed, and that a relocalisation module fired the moment the signal returned. Because the IMU indicates a stationary platform over part of that interval, we filled it in with ZUPT and a motion model.

Windows and chunks. When the dataset is loaded we cut 10-s windows at a stride of 2 s, so that the window starts lie on a fixed period. At every training step we then displace the start position by a random offset drawn from $[-99, +99]$ samples, just under half a second either way, to weaken the dependence on that 2-s period.

Data augmentation. We judged the distribution of the data to be insufficient even so, and therefore augmented it. The transform considers only the relation between acceleration and velocity, never the time axis: scaling the velocity by k scales the acceleration by k as well. Scaling the acceleration, however, scales the bias, the gravity and the noise along with it. We therefore remove the gravity in advance using the ground truth, and estimate the bias by low-pass filtering the difference between an IMU integration model and the ground truth before removing it. To keep the noise from being scaled, we reconstruct the IMU signal up to 2 Hz and scale only that low-frequency part. The menu of scales differs by platform. The drone has by far the widest spread of motion in `train` and `val`, so it is the platform whose distribution most needs filling, and we give it four scales, $k \in \{0.6, 0.8, 1.2, 1.5\}$; the car, the legged platform and the human platform, whose motion is far more confined, use $k = 0.8$ alone. What the menu splits is therefore the data, not the model. Even then a wrong signal can be synthesised and then learned, so we check the consistency with the motion model and mask the infeasible regions; and because a convolutional architecture carries such a signal into its neighbourhood, we mask a further 2 s on either side to leave enough margin.

III. TRAIN AND VALIDATION DETAILS

A. Training and Optimization

Optimisation. Table I lists the settings of the submitted run. Every batch is drawn with the same number of chunks from each of the four platforms, so all four are present at every step and the optimiser never sees a batch dominated by the most numerous platform. Within a platform the chunks are stratified by motion class. We defined those classes ourselves, but they did not appear to make much difference.

Platform-balanced gradients. Weighting the four platform terms equally in the loss does not weight their four gradients equally. The aerial error is roughly four times that of the other platforms, so it dominates backpropagation and the low-error platforms are left underfitted. We therefore rescale

TABLE I
TRAINING HYPERPARAMETERS OF THE SUBMITTED MODEL.

Setting	Value
Optimiser	AdamW
Learning rate	3×10^{-4}
Weight decay	0.05
Warm-up	500 steps, linear
Schedule	cosine decay over 100,000 steps
Batch	40 chunks per platform (1,600 windows)
Platform exponent α	0.85
Marginal entropy	0.05
Commitment	$-0.02 \rightarrow 0.02$ over 3,000 steps
Router z-loss	0.001

TABLE II
DATA USED TO TRAIN THE FINAL SUBMITTED MODEL: TRAJECTORIES AND WINDOWS AS ORIGINAL (+ AUGMENTED COPIES) (§II-C).

Split	Trajectories	Windows
train	395 (+1,262)	81,931 (+124,570)
val	80 (+224)	23,714 (+30,695)
Total	475 (+1,486)	105,645 (+155,265)

each platform term by $\text{term}^{-\alpha}$. The term used as the exponent base is detached and the total loss is renormalised to preserve its magnitude, so the rescaling changes the *direction* of the gradient and not its size. The exponent interpolates between two familiar regimes: $\alpha = 0$ leaves the metric untouched, $\alpha = 1$ equalises the relative contribution of the four platforms, and we use $\alpha = 0.85$.

Gate regularisation. Three terms act on the router. A marginal-entropy term keeps the batch-averaged gate distribution spread across the experts, a commitment term whose weight is ramped from negative to positive keeps the assignment soft early and sharpens it later, and a z-loss keeps the router logits from growing. A platform-classification loss and a supervised contrastive term are present in the code but carry zero weight, so no label ever reaches the router.

Data used. The final submitted model was trained on the official challenge data and on the augmented sets we synthesised from it (§II-C). Table II lists the two official splits that are its only source: 395 trajectories and 81,931 windows of `train`, and 80 trajectories and 23,714 windows of `val`, folded together with the 1,486 augmented copies of §II-C into 105,645 windows before augmentation and 260,910 after. No external public dataset, no data collected or synthesised from any other source, and no pretrained checkpoint was used, and the weights start from random initialisation. The 89 trajectories and 30,644 windows of `test` are read only at inference time and enter no stage of training, and there is no path that feeds predictions back as pseudo-labels.

B. Inference and Complexity

At inference each of the 89 test sequences is passed through the network in one go, and the 12 Hz output is averaged over every 1-s block of 12 steps as in (1). Table III reports the size and the runtime of the model.

TABLE III
MODEL COMPLEXITY AND RUNTIME.

Property	Value
# Params	8.03 M
Training / inference GPU	RTX 5090
Training time	147.5 min
Inference time	15.18 s (all 89 test sequences)
Peak GPU memory	8.43 GB

IV. DEVELOPMENT EXPERIMENTS

We tried three lines of architectural and training changes beyond the submitted model; none is in the final system, and the full account, including the numbers behind each, is in Appendix F.

Platform-specific normalisation. We hypothesised that pooling batch-normalisation statistics across platforms hides features that would otherwise separate them, and tested platform-specific batch normalisation (PSBN), implemented as a soft grouping so that it stays differentiable and never routes on a platform label. PSBN alone made the (unaugmented) score 1.1 % worse than plain BN, but splitting the feature dimension into four normalisation groups (Group PSBN) made it 2.5 % better; we did not carry this into the submission because the residual bias and gravity left by our augmentation (§II-C) makes per-platform statistics only approximately correct, and the gain stopped being consistent once augmentation was on. We kept the plain group normalisation instead.

Feature engineering. Motivated by AirIO’s [8] use of attitude, we fed the network attitude as $\text{so}(3)$, the body-frame gravity directly, a Mahony-filter gravity estimate, a moving average combined with gyroscope integration, and a learned (U-Net) dense gravity predictor. Attitude and raw body-frame gravity both helped; the rest did not, mainly because the estimators themselves are unreliable under the fast motion where the errors concentrate, or because a wrong prediction is less consistent with the raw signal than no prediction at all. What survived into the submitted model is the moving-average branch of §II-B, whose contribution is measured directly in Table V.

Auxiliary supervision. A gravity auxiliary head, a distillation loss against a gravity-input teacher, a body-frame equivariance loss, and heads that predict neighbouring windows $\pm n$ seconds away were all tried and none improved generalisation.

V. EXPERIMENTS

A. Official Challenge Performance

All numbers in Tables IV and VI come from the official scoring service, https://huggingface.co/spaces/Tartan-IMU/imu_odometry_challenge_scoring, and not from a local re-implementation. They were produced by the submission CSV whose MD5 is 4ee059e117393c301c4644a4a6e439f0. AVE and ATE₂₀ are macro-averaged over the four platforms at 25 %

each. The total score is computed as

$$\text{Score} = 0.6 \frac{\text{AVE}}{0.7356} + 0.4 \frac{\text{ATE}_{20}}{3.1160}, \quad (7)$$

dimensionless and lower is better; RTE is reported for analysis and does not enter the ranking. Our public leaderboard score is 0.25392, ranked 5th, against 0.637 for the released TartanIMU baseline on the same split [15]; over all 89 test trajectories the official score is 0.2087 (Table IV, macro average).

Per-Platform Results. Every platform but the drone records an AVE below 0.1 m/s; the drone’s 0.4783 is several times larger (Table IV). This follows from more than its extra degrees of freedom: its speed distribution is far wider. In train/val, the other three platforms rarely exceed 3 m/s, while the drone has many windows above 10 m/s. As a result, the drone alone accounts for more than 60 % of the score.

TABLE IV

PERFORMANCE BY PLATFORM. n IS THE NUMBER OF TEST SEQUENCES, AND THE MACRO AVERAGE OVER THE FOUR PLATFORMS IS THE AGGREGATION THE CHALLENGE SCORES. RTE IS DEFINED IN THE CAPTION OF TABLE VI.

Platform	n	$\text{ATE}_{20} \downarrow$	$\text{AVE} \downarrow$	$\text{RTE} \downarrow$	$\text{Score} \downarrow$
Car	18	0.3390	0.0675	0.2511	0.0986
Legged	16	0.3249	0.0722	0.3535	0.1006
Drone	45	0.9745	0.4783	2.0806	0.5152
Human	10	0.5748	0.0570	0.3017	0.1203
Macro average	89	0.5533	0.1688	0.7467	0.2087

Per-Sequence Results. The errors are concentrated, not spread evenly (Table VI). Two sequences, 29 and 56, have an AVE above 2 m/s; every other sequence stays under 1.0 m/s. No platform label is published per sequence, but the scale of the error and the drone’s standing in Table IV point to two high-speed drone sequences. Sequence 82 is different: its AVE is unremarkable, but its ATE_{20} is the largest in the set, which is what drags its score down. We read this as a human-platform sequence, since the human platform has the lowest AVE of the four in Table IV but the second-highest ATE_{20} , consistent with most of that platform’s ATE loss being concentrated in this one sequence.

B. Ablation Studies

We ran four ablations in total: without the moving average, without FiLM, without data augmentation, and with four heads instead of three.

Data augmentation. Looking first at the results of data augmentation: the car and the legged platform each degrade by 2.7 % in score, but by comparison the drone shows a large improvement. In fact, by AVE, the drone improves by 4.4 %, so the improvement in score comes out to 2.2 %: larger than the degradation on the car and the legged platform, so we can see that it nets out as an improvement.

Gravity conditioning. Dropping the moving-average conditioning branch (which also removes FiLM) costs 5.4 % on the car, 7.0 % on the human platform and 6.5 % on the drone, 5.6 % overall in macro score; only the legged platform is

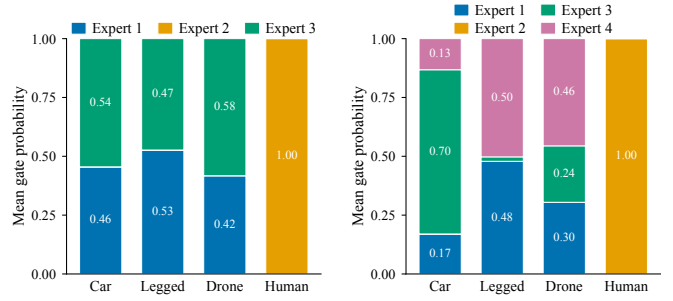


Fig. 3. Mean gate probability per platform, averaged over the test set. **Left:** the submitted model’s three experts. **Right:** the four-heads ablation of §V-B. In both, the human platform routes entirely to one expert and every other platform splits across the rest, but which experts those are and how evenly they are split changes from one configuration to the other. For the full per-window distribution behind these means, see Fig. 6 in Appendix I.

unaffected (−0.2 %), consistent with §F.1. w/o MA is the lowest-scoring of all four ablations. We suspected this was simply parameter count, since it drops an entire encoder, but widening its layer dimension in Appendix G did not recover the gap.

Feature Modulation. FiLM rescales the raw branch’s features with a signal-conditioned (γ, β) pair produced by the moving-average branch (§II-B); without it, the raw branch instead uses canonical normalisation. Losing that conditioning costs 2.8 % overall against Full, most pronounced on the car and the human platform. The legged platform is the exception, where including FiLM actually hurts.

Four heads. Splitting the three experts into four, one per platform instead of letting the car and the legged platform share one (§II-B), costs 3.4 % in macro score: more than dropping augmentation or FiLM, though still less than dropping the branch entirely. It, too, never wins a single row in Table V. We built three heads on the assumption that the car and the legged platform would share a similar expert, and it does score better than four heads. But Fig. 3 shows that, in both three heads and four, every platform but the human one shares its experts, and we could not find a consistent tendency in how. Across all four ablations, the submitted model wins ATE_{20} , AVE and Score outright on the drone, the human platform and the macro average; the one metric that ever prefers an ablation is RTE, and it always prefers no-FiLM.

VI. INSIGHTS AND LESSONS LEARNED

A single model helps, but a single normalisation does not. Our evidence points to a shared backbone with a small amount of platform-aware structure, not either extreme. Pooling batch-normalisation statistics across platforms (§F.1) inflates the variance the wheeled and legged platforms see and can mask the features that separated them cleanly on their own; changing *only* the normalisation scheme moved the score by several percent in Table VII. This raises a question for the TartanIMU [5] authors: when their baseline is fine-tuned online, are the batch-normalisation running statistics updated too? If so, we would expect that shift to compound whatever catastrophic forgetting the fine-tuning already risks.

TABLE V

ABLATIONS AGAINST THE SUBMITTED MODEL (FULL, THE SAME NUMBERS AS TABLE IV), FROM THE OFFICIAL SCORING SERVICE: ATE₂₀ (M), AVE (M/S), RTE (M) AND SCORE PER PLATFORM, WITHOUT DATA AUGMENTATION (W/O AUG.), WITHOUT THE MOVING-AVERAGE CONDITIONING BRANCH (W/O MA), WITHOUT FiLM (W/O FiLM, THE RAW BRANCH USES CANONICAL NORMALISATION INSTEAD), AND WITH FOUR HEADS INSTEAD OF THREE (W/ 4 HEADS). LOWER IS BETTER; THE BEST ENTRY IN EACH ROW IS IN BOLD.

Platform	ATE ₂₀					AVE					RTE					Score				
	Full	w/o aug.	w/o MA	w/o FiLM	w/ 4 heads	Full	w/o aug.	w/o MA	w/o FiLM	w/ 4 heads	Full	w/o aug.	w/o MA	w/o FiLM	w/ 4 heads	Full	w/o aug.	w/o MA	w/o FiLM	w/ 4 heads
Car	0.3390	0.3262	0.3666	0.3904	0.3752	0.0675	0.0663	0.0697	0.0707	0.0690	0.2511	0.2367	0.2652	0.2947	0.2669	0.0986	0.0960	0.1039	0.1077	0.1045
Legged	0.3249	0.3164	0.3280	0.3305	0.3284	0.0722	0.0704	0.0715	0.0695	0.0712	0.3535	0.3402	0.3505	0.3336	0.3442	0.1006	0.0980	0.1004	0.0991	0.1003
Drone	0.9745	1.0127	1.0343	0.9833	1.0252	0.4783	0.5002	0.5101	0.4810	0.4853	2.0806	2.2691	2.1962	1.9300	2.1298	0.5152	0.5380	0.5488	0.5185	0.5274
Human	0.5748	0.5843	0.6257	0.6486	0.6475	0.0570	0.0573	0.0594	0.0606	0.0594	0.3017	0.3039	0.3210	0.3291	0.3256	0.1203	0.1218	0.1287	0.1327	0.1316
Macro	0.5533	0.5599	0.5886	0.5882	0.5941	0.1688	0.1735	0.1777	0.1704	0.1712	0.7467	0.7875	0.7832	0.7219	0.7666	0.2087	0.2134	0.2205	0.2145	0.2159

The bottleneck is the data, not the architecture. Most teams working from the released data alone converged near a public-leaderboard score of 0.28; our augmentation (§II-C) alone moved us to about 0.25, before any architectural change contributed further. That augmentation is exact for the motion it integrates but not for the raw IMU (drone drag and car-ground friction do not scale linearly with k), so moving the score this much with only an approximate signal points to the data, not the network, as the ceiling. Several top teams also added Kalman-filter smoothing on top of a learned model; we read this as the same limit, a classical motion model degrading more gracefully than a learned one under unseen conditions.

What would not transfer. We expect poor generalisation to differently-mounted IMUs: a body frame off by even a small rotation demands a rotation-equivariance never trained for, and that should degrade with the offset rather than fail cleanly. Sensor variation compounds this: our own audit found uncorrected accelerometer bias around 1 m/s² in some sequences, and noise floors differ by IMU, so generalising past this benchmark needs more physical IMUs in training, not just more motion.

Toward resilient inertial odometry. The last two paragraphs are the same difficulty seen twice: an equivariance that only holds if gravity, bias and noise are accounted for, and a bias and noise that drift with motion and wear. What we would try next is closer to resilient odometry [12] than a single frozen model: fine-tune online, per platform and per unit, while another sensor is still healthy enough to supervise it, keeping the batch-normalisation statistics close to their pretrained values so that adapting to one unit does not undo what the rest learned.

A. Feedback to the Organizers

Disclose the sensors and audit the legged platform. The challenge states two IMU models are used, but the data is assembled from several source datasets, so others may be mixed in unstated; please list the source dataset and sensor model per platform. The legged robot is where a nontrivial share of sequences are almost impossible to predict, training or not (Appendix H); our guess is a leg- or head-mounted IMU whose motion is not the torso body-frame motion the labels assume. Several sequences also carry a large stationary bias that a marked static interval could let teams remove.

The metric and the ground truth. As raised in the challenge discussion, please reconsider averaging the window velocity in the world frame rather than the body frame; §II-B and Fig. 2 show why the two disagree under rotation. Some ground truth is also unreliable: our audit (§II-C, Fig. 4) points to relocalisation jumps that we think the official pipeline should catch rather than each team separately. Finally, the platform score means over *sequences* of unequal length, so two short sequences that we attribute to the drone (29 and 56, AVE 2.237 and 2.698 m/s against a platform mean of 0.478 m/s; Table VI) account for nearly a quarter of its total AVE; weighting by sequence length or baseline difficulty would reward broader performance instead.

Set the next edition around the real bottleneck. We believe data, not architecture, is the bottleneck (§VI). An online-learning task (adapting to a platform or sensor from a short calibration run, scored on how little of the shared model it is allowed to forget) would target that directly, in the resilient-odometry [12] direction we think the field should move toward.

VII. CONCLUSION

Through this challenge, we built a single model that unifies multiple platforms without any platform label at inference: a two-branch ConvNeXt-V2 encoder whose moving-average branch modulates the raw branch by FiLM, followed by a bi-GRU and a mixture of three anonymous experts. We believe that feature engineering for the input and coverage of untrained data, more than the deep learning model itself, have the greatest effect on performance. The model is still bounded by the motion and sensors its training data covers, and broader data coverage and online adaptation to new platforms and sensors are the directions we find most promising.

ACKNOWLEDGMENT

The authors thank Claude AI for assistance with code and documentation.

REFERENCES

- [1] C. Chen, X. Lu, A. Markham, and N. Trigoni, “IONet: Learning to cure the curse of drift in inertial odometry,” in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 32, no. 1, 2018.

- [2] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust neural inertial navigation in the wild: Benchmark, evaluations, and new methods,” in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2020, pp. 3146–3152.
- [3] W. Liu, D. Caruso, E. Ilg, J. Dong, A. I. Mourikis, K. Daniilidis, V. Kumar, and J. Engel, “TLIO: Tight learned inertial odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [4] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial deep orientation-estimation and localization,” in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 35, no. 7, 2021, pp. 6128–6137.
- [5] S. Zhao, S. Zhou, R. Blanchard, Y. Qiu, W. Wang, and S. Scherer, “Tartan IMU: A Light Foundation Model for Inertial Positioning in Robotics,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 22520–22529.
- [6] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [7] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [8] Y. Qiu, C. Xu, Y. Chen, S. Zhao, J. Geng, and S. Scherer, “AirIO: Learning Inertial Odometry with Enhanced IMU Feature Observability,” *IEEE Robotics and Automation Letters*, vol. 10, no. 9, pp. 9368–9375, 2025.
- [9] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11976–11986.
- [10] S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, “ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 16133–16142.
- [11] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [12] S. Zhao, S. Zhou, Y. Zhang, J. Zhang, C. Wang, W. Wang, and S. Scherer, “Resilient odometry via hierarchical adaptation,” *Science Robotics*, vol. 10, no. 109, p. eadv1818, 2025.
- [13] W.-G. Chang, T. You, S. Seo, S. Kwak, and B. Han, “Domain-specific batch normalization for unsupervised domain adaptation,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 7354–7362.
- [14] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2012, pp. 573–580.
- [15] “TartanIMU,” GitHub repository, accessed Sep. 2026. [Online]. Available: <https://github.com/superslam/TartanIMU>

TABLE VI

PER-SEQUENCE CHALLENGE RESULTS OVER THE 89 TEST SEQUENCES. ATE, AVE AND RTE ARE PRODUCED BY THE OFFICIAL PER-SEQUENCE SCORING SERVICE WHOSE URL IS GIVEN IN §V-A. **RTE** (RELATIVE TRAJECTORY ERROR, M) FOLLOWS THE DEFINITION OF STURM *et al.* [14] AND IS THE METRIC USED IN AIRIO [8]: OVER SLIDING WINDOWS OF A FIXED DURATION Δt , IT IS THE RMSE OF THE DIFFERENCE BETWEEN THE PREDICTED AND THE GROUND-TRUTH RELATIVE DISPLACEMENT, $RTE = \sqrt{\frac{1}{n} \sum_i \|(\hat{\mathbf{p}}_{i+\Delta t} - \hat{\mathbf{p}}_i) - (\mathbf{p}_{i+\Delta t} - \mathbf{p}_i)\|^2}$. UNLIKE ATE IT MEASURES *local* DRIFT AND IS INSENSITIVE TO ACCUMULATED GLOBAL OFFSET. THE WINDOW LENGTH IS FIXED TO $\Delta t = 5$ S FOR ALL PLATFORMS: AT $\Delta t = 1$ S THE METRIC IS NEARLY A RESTATEMENT OF AVE, WHILE THE SHORTEST AERIAL SEQUENCES ARE ONLY 19 S LONG, WHICH BOUNDS Δt FROM ABOVE.

Seq.	ATE ₂₀	AVE	RTE	Seq.	ATE ₂₀	AVE	RTE	Seq.	ATE ₂₀	AVE	RTE
1	0.859	0.397	1.930	31	0.794	0.390	1.737	61	1.499	0.444	1.971
2	0.503	0.322	1.162	32	0.214	0.067	0.174	62	0.080	0.014	0.046
3	0.674	0.286	1.141	33	0.182	0.042	0.174	63	0.555	0.075	0.374
4	0.459	0.255	0.737	34	1.176	0.292	1.484	64	0.452	0.064	0.304
5	0.587	0.280	1.307	35	1.347	0.382	1.554	65	0.103	0.036	0.109
6	0.682	0.088	0.767	36	1.015	0.410	2.043	66	1.599	0.249	1.794
7	0.936	0.257	1.209	37	0.635	0.939	4.967	67	0.127	0.030	0.144
8	0.472	0.038	0.205	38	0.710	0.892	4.234	68	0.211	0.031	0.112
9	0.481	0.185	0.692	39	0.338	0.162	0.498	69	0.755	0.095	0.507
10	1.943	0.153	0.980	40	0.843	0.385	2.072	70	0.558	0.069	0.307
11	1.146	0.775	1.776	41	1.153	0.519	2.530	71	0.250	0.062	0.206
12	1.113	0.468	2.078	42	0.403	0.073	0.292	72	1.148	0.550	1.437
13	0.190	0.033	0.167	43	0.888	0.267	1.138	73	0.618	0.117	0.510
14	0.614	0.365	1.303	44	0.233	0.049	0.313	74	0.652	0.277	0.999
15	1.415	0.089	0.521	45	0.080	0.051	0.124	75	0.579	0.381	1.260
16	0.142	0.031	0.203	46	0.278	0.053	0.220	76	0.219	0.051	0.181
17	0.556	0.396	1.433	47	0.707	0.358	1.219	77	1.205	0.665	2.033
18	1.306	0.489	2.330	48	0.283	0.015	0.098	78	0.119	0.022	0.081
19	0.825	0.343	1.564	49	0.709	0.283	1.152	79	0.507	0.220	1.029
20	0.142	0.047	0.344	50	0.470	0.199	0.883	80	1.363	0.432	2.808
21	0.748	0.231	1.045	51	0.097	0.034	0.104	81	0.421	0.140	0.364
22	0.615	0.355	1.250	52	0.167	0.070	0.195	82	3.518	0.349	2.339
23	1.079	0.376	2.112	53	0.420	0.281	0.531	83	0.452	0.071	0.383
24	0.457	0.047	0.220	54	0.204	0.047	0.175	84	0.500	0.056	0.326
25	0.087	0.026	0.075	55	0.644	0.286	1.339	85	0.072	0.033	0.102
26	0.226	0.075	0.188	56	2.669	2.698	11.378	86	0.379	0.029	0.170
27	0.480	0.063	0.293	57	1.017	0.438	2.298	87	0.586	0.175	0.839
28	0.793	0.242	1.247	58	0.233	0.047	0.247	88	1.139	0.468	2.437
29	2.500	2.237	9.937	59	0.277	0.094	0.269	89	0.087	0.019	0.061
30	0.748	0.404	1.698	60	0.184	0.067	0.235				

APPENDIX

A. Per-Sequence Results (Full Table)

The full per-sequence results are given in Table VI.

B. Reproducibility and Submission Artifacts

Kaggle team name: **CoCEL @POSTECH**. Submission CSV: `submission_CoCEL_POSTECH.csv` (submitted to Kaggle as `G1_p2_convgate_nofilm.csv`), md5 = `4ee059e117393c301c4644a4a6e439f0` (§V-A).

Code repository: https://huggingface.co/pash03023/TartanIMU-CoCEL_POSTECH, commit `88040d6984d0f2fe977a5eb378d903af735b9ade`.

Trained checkpoint: https://huggingface.co/pash03023/TartanIMU-CoCEL_POSTECH/blob/main/CoCEL_POSTECH.pt, md5 = `ae40194178468ccac395138e3bbc44f4`. The reported score is a single run, from a single training run (random seed 0) with no seed averaging, checkpoint averaging or ensembling (§III-A, §III-B).

Compliance declaration.

- 1) **Single model, one shared set of weights.** Yes. All four embodiments are predicted by one `state_dict`.

There is no per-platform checkpoint and no dispatch to separately trained experts.

- 2) **Platform classification or routing at inference.** None. The only routing is the internal gate of §II-B, a light-weight convolution over the raw signal that never sees a platform label. The inference input is the raw 6-axis IMU alone.
- 3) **Ensembling.** None. No model ensemble, no checkpoint averaging or soup, and no test-time augmentation: one forward pass per trajectory with one set of weights.
- 4) **Test-set leakage.** `test` is read only at inference. It enters no stage of training, no statistic is fitted on it, and no prediction is fed back as a pseudo-label. The checkpoint is fixed by a rule rather than by a score, since `train` and `val` are both in the fitting set and the last step is taken as is. Among our candidate configurations we adopted the one with the best public leaderboard score.
- 5) **Development versus submitted system.** They are the same; nothing was changed for the final submission. The ablations of §IV are separate runs without aug-

mentation, fitted on `train` alone.

C. Final Predictions

The predictions are reproduced by running the inference script of §D on the checkpoint linked there; MD5 of the submitted CSV:

4ee059e117393c301c4644a4a6e439f0 (§V-A). No post-processing (TTA, smoothing, rescaling, ensembling or soup) is applied after the network output (§III-B). The submitted CSV is available at https://huggingface.co/pash03023/TartanIMU-CoCEL_POSTECH/blob/main/submission_CoCEL_POSTECH.csv.

D. Model and Inference Code

Code repository: https://huggingface.co/pash03023/TartanIMU-CoCEL_POSTECH, commit 88040d6984d0f2fe977a5eb378d903af735b9ade.

Trained checkpoint: https://huggingface.co/pash03023/TartanIMU-CoCEL_POSTECH/blob/main/CoCEL_POSTECH.pt, md5 = ae40194178468ccac395138e3bbc44f4.

The submitted CSV is reproduced with the repository’s `predict.py`:

```
pip install -r requirements.txt
python predict.py \
  --data /path/to/challenge_data \
  --weights CoCEL_POSTECH.pt \
  --out submission_CoCEL_POSTECH.csv
```

Every preprocessing setting is read from the checkpoint. Dependencies are pinned in `requirements.txt` (Python 3.10, PyTorch 2.13.0 with CUDA 13.0), and CUDA is required for a bit-exact match.

E. World- Versus Body-Frame Loss Under L1

§II-B states that the 1-s loss is taken in the world frame rather than the body frame, and that this matters only because the loss is L1. Here is why.

Let $\mathbf{B}\mathbf{e} \in \mathbb{R}^3$ be the error between the predicted and the ground-truth body-frame velocity at a given instant, and let $\mathbf{W}\mathbf{e} = \mathbf{R}\mathbf{B}\mathbf{e}$ be the same error rotated into the world frame by the ground-truth orientation $\mathbf{R} \in SO(3)$, an orthogonal matrix ($\mathbf{R}^\top \mathbf{R} = \mathbf{I}$).

L2 is frame-invariant. For the squared Euclidean (L2) norm,

$$\begin{aligned} \|\mathbf{W}\mathbf{e}\|_2^2 &= (\mathbf{R}\mathbf{B}\mathbf{e})^\top (\mathbf{R}\mathbf{B}\mathbf{e}) \\ &= \mathbf{B}\mathbf{e}^\top \mathbf{R}^\top \mathbf{R} \mathbf{B}\mathbf{e} = \mathbf{B}\mathbf{e}^\top \mathbf{B}\mathbf{e} = \|\mathbf{B}\mathbf{e}\|_2^2, \end{aligned} \quad (8)$$

using only $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$. An MSE loss would therefore give exactly the same value whether the 1-s term is formed in the body frame or the world frame, and the design choice in §II-B would make no difference at all.

L1 is not. Our loss instead sums the per-axis absolute error, $\|\mathbf{x}\|_1 = \sum_{j=1}^3 |x_j|$. Writing out the world-frame version,

$$\|\mathbf{W}\mathbf{e}\|_1 = \sum_{j=1}^3 \left| \sum_{k=1}^3 R_{jk} \mathbf{B}e_k \right|, \quad (9)$$

each term now mixes all three body-frame components inside one absolute value, and there is no orthogonality identity that reduces (9) back to $\|\mathbf{B}\mathbf{e}\|_1 = \sum_k |\mathbf{B}e_k|$: that reduction holds only for the signed-permutation matrices, a finite subgroup of $SO(3)$, not for a general rotation. This is exactly the asymmetry §II-B is written to avoid: forming the 1-s term in the body frame would let the loss depend on how the error happens to be split across the body’s own axes, rather than on its true magnitude, so we form it in the world frame instead.

F. Extended Development Experiments

The condensed account in §IV summarises three lines of development work; the full reasoning, the failed variants, and the numbers behind each is below.

TABLE VII
NORMALISATION VARIANTS, TRAINED ON `train` WITHOUT AUGMENTATION AND SCORED ON `val`. LOWER IS BETTER; THE BEST ENTRY IN EACH ROW IS IN BOLD.

	BN	PSBN	Group BN	Group PSBN
Car	0.097	0.092	0.093	0.092
Legged	0.087	0.083	0.087	0.085
Drone	0.274	0.285	0.281	0.263
Human	0.068	0.069	0.067	0.067
AVE	0.1351	0.1367	0.1341	0.1305
Score	0.17638	0.17835	0.17499	0.17201

1) Soft Grouped Platform-specific Batch Normalization:

Hypothesis. We looked at batch normalisation (BN). Our hypothesis was that a BN layer whose statistics pool several platforms weakens the ability to tell those platforms apart. The degrees of freedom are not the same: the car and the legged platform are held to the ground and carry no gravity component outside z , whereas the drone and the human platform are free to tilt. Following domain-specific BN (DSBN) [13], which keeps one normalisation per domain, we tried a platform-specific BN (PSBN).

Soft grouping. The rules do not allow this to be implemented as routing, so we group softly. BN updates its running mean and variance by an exponential moving average during training. We first pretrain the gate; then the encoder feature $\mathbf{F}_{\text{enc}}$ is distributed as $\pi_p \mathbf{F}_p$ over $p \in \{\text{drone, car, human, legged}\}$, each branch applies its own affine transformation, and the mean and the variance are updated within that branch. The per-platform mean and variance are then subtracted from $\mathbf{F}_{\text{enc}}$ and the features are gathered again by the weighted mean of (3). Because each branch has moved the feature to the mean of its own platform, we use four heads to keep them apart and apply (3) once more for the final output. Only the normalisation and the heads differ; the network that extracts the features is shared, and the soft gate keeps the whole path differentiable.

What PSBN did. We evaluated every variant without augmentation, training on `train` and evaluating on `val`. PSBN lowered AVE for the car and the legged platform but raised it for the drone and the human platform, and the score came out 1.1 % worse than BN (Table VII).

TABLE VIII

FEATURE-ENGINEERING VARIANTS FOR THE MA BRANCH’S INPUT (§F.2), TRAINED FOR 20,000 STEPS ON `TRAIN` WITHOUT AUGMENTATION AND SCORED ON `VAL`. THE **GT** ROWS REPLACE THE BRANCH’S INPUT WITH GROUND TRUTH THAT IS UNAVAILABLE AT INFERENCE AND SERVE ONLY AS AN ORACLE UPPER BOUND; THE FOUR PLATFORM COLUMNS ARE AVE, AND AVE, ATE₂₀ AND SCORE ARE MACRO-AVERAGED OVER THE FOUR PLATFORMS. LOWER IS BETTER; THE BEST ENTRY IN EACH COLUMN IS IN BOLD.

Variant	MA branch input	AVE by platform				Macro		
		Car	Legged	Drone	Human	AVE	ATE ₂₀	Score
Ours	low-pass-filtered acceleration	0.090	0.083	0.272	0.067	0.1302	0.4932	0.16951
Ours w/o MA	no branch	0.096	0.084	0.279	0.068	0.1344	0.5277	0.17735
Ours w/ ^B g (GT)	ground-truth body-frame gravity	0.086	0.061	0.227	0.052	0.1111	0.4181	0.14431
Ours w/ $\mathfrak{so}(3)$ (GT)	ground-truth full $\mathfrak{so}(3)$ attitude	0.089	0.061	0.211	0.063	0.1134	0.4364	0.14853

Grouped normalisation. We read that as a loss of statistics. A BN layer sees all four platforms at once, whereas PSBN leaves each branch with a quarter of the samples. Enlarging the batch would restore the count but would no longer be the same comparison, so we instead borrowed from group normalisation and split the feature dimension by position into four groups, normalising over those groups. As a side effect the affine parameters of the normalisation shrink to a quarter. Group PSBN beats plain BN by 2.5 % in score, and its gain over PSBN comes almost entirely from the drone, where AVE falls from 0.285 to 0.263. Grouping BN alone, without the per-platform statistics, recovers only 0.8 %, so most of the gain comes from the two changes together rather than from either one.

Why we do not use it. Group PSBN scored better, yet the submitted model does not use it and relies on group normalisation alone. The reason is the data augmentation of §II-C. There we scale the signal by k and strip the bias and the gravity beforehand so that neither is amplified along with it, but that removal is not exact. On an augmented root the per-platform statistics therefore rest on a signal that is only approximately corrected, and the gain of Group PSBN stopped being consistent. We submitted the variant with group normalisation, which gave the best score among our experiments.

2) Feature Engineering:

Motivation. AirIO [8] raises accuracy by taking the attitude as an input, and we tried a number of features in that spirit; the moving average used in the submitted model comes from the same reasoning. All of these experiments were run on `train` and `val` only. We tried (1) the attitude as $\mathfrak{so}(3)$, (2) the body-frame gravity, (3) a gravity estimate from a Mahony filter, (4) the moving average combined with gyroscope integration, and (5) a U-Net that predicts a dense gravity signal.

Attitude and yaw. Representing the attitude as $\mathfrak{so}(3)$, as AirIO does, improved the score, as one would expect. We doubted that yaw should matter for the motion itself, and suspected the gain came from the rotation supplying a global positional relation. Perturbing yaw, pitch and roll separately on the validation split bore that out: the loss under a yaw perturbation was by far the largest.

Gravity. To see whether gravity alone is enough, we fed the body-frame gravity directly, and it scored better than

TABLE IX

W/O MA AT TWO CAPACITIES, FROM THE OFFICIAL SCORING SERVICE. LIGHT IS THE W/O MA OF TABLE V (4.68 M PARAMETERS); HEAVY WIDENS ITS ENCODER TO 8.17 M, ABOVE THE SUBMITTED MODEL’S 8.03 M. THE SUBMITTED MODEL ITSELF IS NOT REPEATED HERE. LOWER IS BETTER; THE BEST OF THE TWO IN EACH ROW IS IN BOLD.

Platform	ATE ₂₀		AVE		Score	
	Light (4.68 M)	Heavy	Light	Heavy	Light	Heavy (8.17 M)
Car	0.3666	0.3484	0.0697	0.0683	0.1039	0.1004
Legged	0.3280	0.3257	0.0715	0.0720	0.1004	0.1006
Drone	1.0343	1.0428	0.5101	0.5120	0.5488	0.5515
Human	0.6257	0.6958	0.0594	0.0638	0.1287	0.1414
Macro	0.5886	0.6032	0.1777	0.1790	0.2205	0.2235

$\mathfrak{so}(3)$. Every platform improved, most on the legged and the human platform; even the car and the legged platform, held to the ground, gained. Estimating that gravity with a Mahony filter changed almost nothing: the filter tracks well under slow motion but not under the fast motion of the drone, which is exactly where the accuracy is lost. Adding gyroscope integration to the moving average, so as to describe the gravity more finely by accounting for the instantaneous rotation, also made little difference. Table VIII gives the numbers behind both paragraphs above.

Learned gravity. We then trained a network to predict the gravity and used its output as an input, with a U-Net that upsamples what the 1-D CNN has downsampled so that the prediction is available at 200 Hz. The result was equal or worse, for two reasons: the predicted gravity does not carry over to unseen motion, and the gravity is already recoverable from the raw IMU, so a wrong prediction is inconsistent with the signal the network reads in any case. We also formed the Coriolis term as the cross product of ^B**a** and ^B**w** and added it as an input, with the same outcome. We therefore rejected explicit feature engineering.

3) *Auxiliary Tasks:* Where the previous subsection added features to the input, this one adds supervision. None of the four attempts below survived into the submitted model.

A gravity head. Instead of feeding the gravity as an input we predicted it from an auxiliary head. It was withdrawn for the reason already given in §F.1: under augmentation the gravity label is not accurate enough to supervise against.

Distillation. Since the moving average only imitates the body-frame gravity, we took a network that receives the true

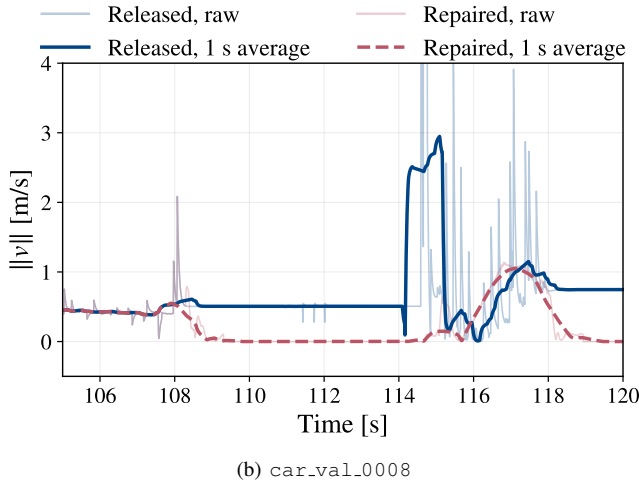
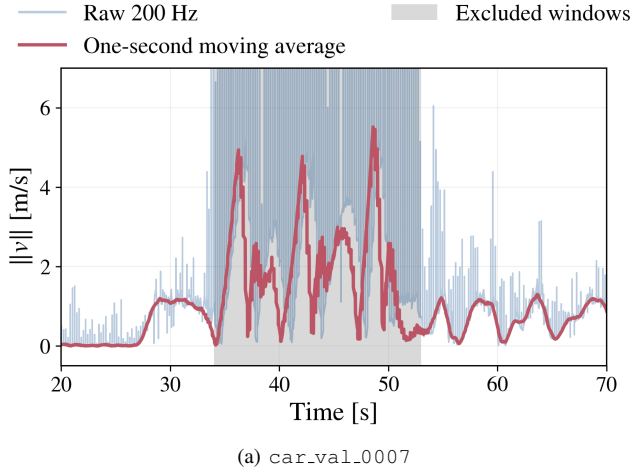


Fig. 4. Two infeasible ground-truth labels on the car platform, each shown over an excerpt of its sequence. (a) The released 200 Hz speed spikes far beyond what the platform can reach, and the 1-s mean it feeds is visibly inflated over the affected stretch; the shaded windows are the ones we drop from the loss. (b) The released track holds a constant speed through a long gap and then jumps when it resumes, which we read as a failed localisation followed by a relocalisation. The repaired curve replaces that stretch with a zero-velocity update and a motion model. Both panels are clipped on the vertical axis.

body-frame gravity as its input as a teacher and distilled from it, among others with a centred kernel alignment (CKA) loss. The student did not improve.

Body-frame equivariance. We rotated the body frame about the gravity direction and added an equivariance loss between the two views. The gain was not meaningful.

Neighbour prediction. We attached heads identical to the velocity head that predict the windows $\pm n$ seconds away, and weighted their terms by a discount factor. Training converged, but generalisation did not improve.

G. Is w/o MA Just Smaller?

Removing the moving-average branch of Fig. 1 for w/o MA (§II-B, Table V) also removes an entire encoder, so the variant is smaller: 4.68 M parameters against the submitted model’s 8.03 M, a 41.7% cut. That confound makes it unclear whether w/o MA loses to the branch itself or simply to having fewer weights. To separate the two, we widened the

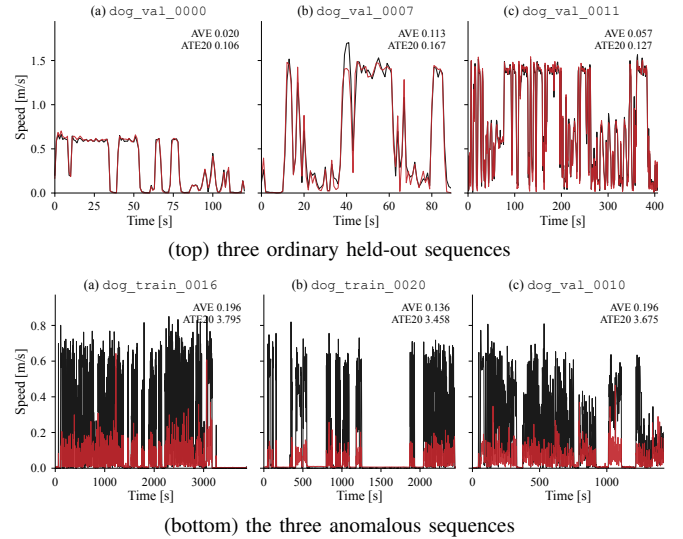


Fig. 5. Predicted (red) against ground-truth (black) speed for held-out legged sequences never seen in training. The three anomalous sequences, dog_train_0016, dog_train_0020 and dog_val_0010, are an order of magnitude worse in ATE₂₀ than the ordinary sequences above, and the prediction does not track the ground truth at all.

remaining encoder’s layer dimension by roughly $1.5\times$ and reran w/o MA at a matched (in fact slightly larger) capacity: 8.17 M parameters, 0.14 M above the submitted model. We call the original w/o MA *Light* and this one *Heavy*.

Heavy is not simply Light with the deficit made up: it wins outright on the car, where every metric improves, but it is worse than Light on the macro score, and on the human platform it is worse on every metric by a wide margin despite carrying more parameters than the submitted model itself. Extra capacity in the remaining encoder does not recover what the moving-average branch was contributing; the human platform, which has the fewest test sequences of the four, is also where matching capacity hurts the most, consistent with the extra parameters being spent on fitting noise rather than on anything the branch used to supply. We read this as evidence that w/o MA’s loss in Table V is architectural and not merely a parameter-count confound.

H. Anomalous Legged Sequences

We noticed that dog_val_0010 scored unusually poorly and used an AI assistant (Claude) to help investigate why. The finding it confirmed was that the gyroscope reading and the ground-truth orientation disagree even over short intervals: integrating the gyroscope does not reproduce the ground-truth rotation. We then asked it to check for other sequences with the same symptom and found two more, dog_train_0016 and dog_train_0020.

To check whether these three are simply hard to predict in general, we retrained excluding them, together with the usual held-out split, and evaluated on sequences the model had never trained on. Fig. 5 shows the result: the three sequences are predicted far worse than every other held-out sequence, which the model tracks reasonably well even without having trained on it directly.

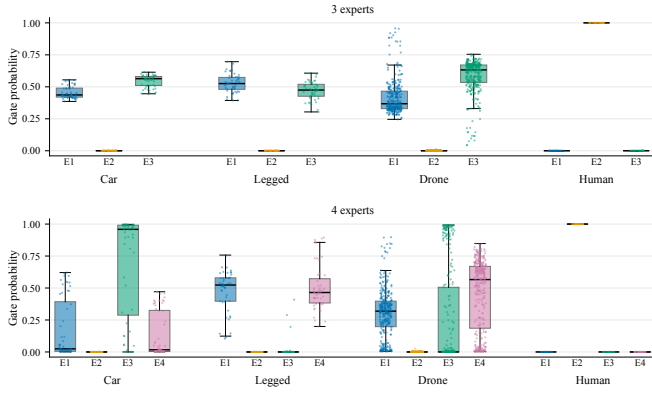


Fig. 6. Per-window gate probability by platform and expert, for the submitted three-expert model (top) and the four-heads ablation (bottom). Each point is one test window; boxes show the quartiles.

Looking at world position and velocity, all three share something the others do not: their ground truth was generated at 5 Hz rather than 200 Hz. That points to a different, coarser generation pipeline for these three specifically, though, unlike `car_val_0007` and `car_val_0008` (§II-C, Fig. 4), the values themselves are not physically infeasible. Our guess is mounting: a leg- or head-mounted IMU on a legged platform moves far more than the torso does, and if the ground truth is generated for the torso and only reprojected onto the IMU’s location through a fixed extrinsic, that reprojection would not track the sensor’s actual motion. We nonetheless kept these three sequences in training; excluding them made validation performance worse, not better.

I. Gate Weight Distributions

Fig. 3 (§V-B) shows only the per-platform mean gate probability. Fig. 6 gives the full per-window distribution behind those means, for both the submitted three-expert model and the four-heads ablation.